

RDFS and OWL Inference

Simon Frost

Overview

RDFLib.jl supports forward-chaining inference using RDFS (RDF Schema) and OWL (Web Ontology Language) rules. This vignette demonstrates how to compute closures and check entailment.

```
using RDFLib
ex = Namespace("http://example.org/")
```

```
Namespace("http://example.org/")
```

RDFS Inference

RDFS defines a vocabulary for class hierarchies and property domains/ranges. The RDFS closure computes all implied triples.

Subclass Reasoning

```
g = parse_rdf("""
    @prefix ex: <http://example.org/> .
    @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

    ex:Dog rdfs:subClassOf ex:Mammal .
    ex:Mammal rdfs:subClassOf ex:Animal .
    ex:Animal rdfs:subClassOf ex:LivingThing .

    ex:fido a ex:Dog .
""", TurtleFormat())
```

```

println("Before inference: ${length(g)} triples")

# Compute RDFS closure
inferred = rdfs_closure(g)
println("After RDFS closure: ${length(inferred)} triples")

# Check: Is Fido an Animal?
is_animal = any(t ->
  t.subject == ex("fido") && t.predicate == RDF.type && t.object == ex("Animal"),
  inferred)
println("\nFido is an Animal: $is_animal")

# What types does Fido have?
println("\nFido's types:")
for t in triples(inferred, (ex("fido"), RDF.type, nothing))
  println("  ", t.object)
end

```

Before inference: 4 triples
 After RDFS closure: 10 triples

Fido is an Animal: true

Fido's types:
 URIRef("http://example.org/LivingThing")
 URIRef("http://example.org/Animal")
 URIRef("http://example.org/Dog")
 URIRef("http://example.org/Mammal")

Domain and Range Inference

```

g2 = parse_rdf("""
  @prefix ex: <http://example.org/> .
  @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

  ex:hasParent rdfs:domain ex:Person .
  ex:hasParent rdfs:range ex:Person .

  ex:alice ex:hasParent ex:bob .
""", TurtleFormat())

```

```

inferred = rdfs_closure(g2)
println("Inferred types:")
for t in triples(inferred, (nothing, RDF.type, ex("Person")))
    println("  $(t.subject) a Person")
end

```

```

Inferred types:
  URIRef("http://example.org/alice") a Person
  URIRef("http://example.org/bob") a Person

```

SubProperty Reasoning

```

g3 = parse_rdf("""
  @prefix ex: <http://example.org/> .
  @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

  ex:hasMother rdfs:subPropertyOf ex:hasParent .
  ex:hasParent rdfs:subPropertyOf ex:hasAncestor .

  ex:alice ex:hasMother ex:carol .
""", TurtleFormat())

inferred = rdfs_closure(g3)

has_parent = any(t ->
  t.subject == ex("alice") && t.predicate == ex("hasParent") && t.object == ex("carol"),
  inferred)
has_ancestor = any(t ->
  t.subject == ex("alice") && t.predicate == ex("hasAncestor") && t.object == ex("carol"),
  inferred)

println("alice hasParent carol:  $has_parent")
println("alice hasAncestor carol: $has_ancestor")

```

```

alice hasParent carol:  true
alice hasAncestor carol: true

```

OWL Inference

OWL provides richer class constructors and property axioms.

```
g4 = parse_rdf("""
    @prefix ex: <http://example.org/> .
    @prefix owl: <http://www.w3.org/2002/07/owl#> .
    @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

    ex:Person a owl:Class .
    ex:Human owl:equivalentClass ex:Person .

    ex:alice a ex:Human .
""", TurtleFormat())

inferred = owl_closure(g4)
is_person = any(t ->
    t.subject == ex("alice") && t.predicate == RDF.type && t.object == ex("Person"),
    inferred)
println("alice is a Person (via equivalentClass): $is_person")
```

alice is a Person (via equivalentClass): true

Inverse Properties

```
g5 = parse_rdf("""
    @prefix ex: <http://example.org/> .
    @prefix owl: <http://www.w3.org/2002/07/owl#> .

    ex:hasChild owl:inverseOf ex:hasParent .
    ex:alice ex:hasChild ex:bob .
""", TurtleFormat())

inferred = owl_closure(g5)
has_parent = any(t ->
    t.subject == ex("bob") && t.predicate == ex("hasParent") && t.object == ex("alice"),
    inferred)
println("bob hasParent alice (via inverseOf): $has_parent")
```

bob hasParent alice (via inverseOf): true

Symmetric Properties

```
g6 = parse_rdf("""
    @prefix ex: <http://example.org/> .
    @prefix owl: <http://www.w3.org/2002/07/owl#> .

    ex:knows a owl:SymmetricProperty .
    ex:alice ex:knows ex:bob .
""", TurtleFormat())

inferred = owl_closure(g6)
symmetric = any(t ->
    t.subject == ex("bob") && t.predicate == ex("knows") && t.object == ex("alice"),
    inferred)
println("bob knows alice (via symmetry): $symmetric")
```

bob knows alice (via symmetry): true

Transitive Properties

```
g7 = parse_rdf("""
    @prefix ex: <http://example.org/> .
    @prefix owl: <http://www.w3.org/2002/07/owl#> .

    ex:partOf a owl:TransitiveProperty .
    ex:cell ex:partOf ex:tissue .
    ex:tissue ex:partOf ex:organ .
    ex:organ ex:partOf ex:body .
""", TurtleFormat())

inferred = owl_closure(g7)
println("Transitive partOf:")
for t in triples(inferred, (ex("cell"), ex("partOf"), nothing))
    println("    cell partOf $(t.object)")
end
```

Transitive partOf:

```
cell partOf URIRef("http://example.org/organ")
cell partOf URIRef("http://example.org/body")
cell partOf URIRef("http://example.org/tissue")
```

Entailment Checking

Test whether one graph logically follows from another:

```
premise = parse_rdf("""
    @prefix ex: <http://example.org/> .
    @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
    ex:Cat rdfs:subClassOf ex:Animal .
    ex:whiskers a ex:Cat .
""", TurtleFormat())

# Apply RDFS closure first, then check entailment for a specific triple
closed = rdfs_closure(premise)
conclusion_triple = Triple(URIRef("http://example.org/whiskers"), RDF.type, URIRef("http://example.org/whiskers"))
println("Premise entails conclusion: ", entails(closed, conclusion_triple))
```

Premise entails conclusion: true

Building OWL Ontologies with InfixOWL

RDFLib.jl provides a DSL for building OWL class definitions:

```
g8 = RDFGraph()
bind!(g8, "ex", ex)
bind!(g8, "owl", Namespace("http://www.w3.org/2002/07/owl#"))

# Declare an ontology
owl_ontology!(g8, ex("myOntology"); label="Example Ontology")

# Define classes
animal = OWLClass(ex("Animal"), g8)
mammal = OWLClass(ex("Mammal"), g8)
dog = OWLClass(ex("Dog"), g8)

# Class hierarchy
subclass_of!(mammal, animal)
subclass_of!(dog, mammal)

# Properties
has_name = OWLDatatypeProperty(ex("hasName"), g8)
has_owner = OWLObjectProperty(ex("hasOwner"), g8)
```

```
println("OWL ontology ($(length(g8)) triples):")
println(serialize(g8, TurtleFormat()))
```

```
OWL ontology (9 triples):
@prefix ex: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:Animal a owl:Class .

ex:Dog a owl:Class ;
    rdfs:subClassOf ex:Mammal .

ex:Mammal a owl:Class ;
    rdfs:subClassOf ex:Animal .

ex:hasName a owl:DatatypeProperty .

ex:hasOwner a owl:ObjectProperty .

ex:myOntology a owl:Ontology ;
    rdfs:label "Example Ontology" .
```